MusicTwins – Documento Técnico de Desarrollo

Documento Técnico Completo para Equipos de Desarrollo | Stack: Next.js + Node.js + PostgreSQL | Versión 1.0 – Marzo 2026

1. Resumen Ejecutivo

MusicTwins es una aplicacion web social enfocada en la actividad musical de los usuarios. Permite compartir y visualizar en tiempo real lo que los amigos estan escuchando en Spotify, reaccionar con emojis a sus reproducciones, agregar notas sobre canciones y mantener conversaciones 1 a 1 vinculadas o no a canciones especificas del feed.

El sistema se autentica exclusivamente mediante OAuth 2.0 con Spotify (Authorization Code / PKCE), sin credenciales propias. La aplicacion persiste la actividad musical de los usuarios en una base de datos PostgreSQL y la presenta en un feed cronologico social. El objetivo de negocio principal es convertir la experiencia de escucha individual en una experiencia compartida entre amigos, creando engagement en torno a la musica.

Usuarios objetivo: personas que usan Spotify activamente y desean compartir su actividad musical con amigos dentro de un entorno social propio. Alcance inicial: autenticacion con Spotify, feed social de reproducciones, reacciones, notas, sistema de amigos y mensajeria 1 a 1 en tiempo real via WebSockets. Fase 2 (fuera del alcance inicial): integracion con Apple Music, notificaciones push, recomendaciones musicales.

2. Requisitos Funcionales (RF)

2.1 Autenticacion y Usuarios

RF-01 | El sistema debe permitir iniciar sesion usando el boton Continuar con Spotify. | Alta | Al hacer clic, el usuario es redirigido a la pagina oficial de Spotify y regresa autenticado a MusicTwins.
RF-02 | El sistema debe crear un perfil interno de usuario a partir de los datos de Spotify. | Alta | Tras el primer login se crea un registro en la tabla users con spotify_id, display_name, email y avatar_url.
RF-03 | El sistema debe mantener la sesion del usuario mediante JWT. | Alta | Despues de autenticarse, el usuario puede recargar la pagina y sigue logueado mientras el token sea valido.
RF-04 | El sistema debe permitir cerrar sesion. | Media | Al hacer clic en Cerrar sesion, se eliminan los tokens del cliente y se redirige a la pantalla de bienvenida.

2.2 Integracion con Spotify

RF-05 | El sistema debe implementar Authorization Code/PKCE de Spotify para obtener tokens. | Alta | Tras el login, el backend intercambia el code por access_token y refresh_token y los guarda en streaming_accounts con su fecha de expiracion.
RF-06 | El sistema debe refrescar el access_token de forma automatica al expirar. | Alta | Cuando una llamada a Spotify retorna error por token expirado, el backend obtiene uno nuevo y reintenta sin que el usuario lo perciba.
RF-07 | El sistema debe obtener el track currently playing del usuario. | Media | Existe un endpoint interno que devuelve el track actual con trackId, name, artist, album, imageUrl e isPlaying.
RF-08 | El sistema debe obtener el historial reciente de reproducciones del usuario. | Media | Endpoint interno que devuelve las ultimas 20 reproducciones con track, artista y played_at.
RF-09 | El sistema debe persistir los eventos de reproduccion relevantes en playback_events. | Alta | Los eventos traidos de Spotify para el usuario y sus amigos se almacenan con user_id, track_id, played_at y metadata basica.

2.3 Amigos y Social Graph

RF-10 | El sistema debe permitir buscar usuarios por nombre o alias. | Alta | Desde el buscador, ingresar texto devuelve una lista paginada de usuarios coincidentes excluyendo al usuario actual.
RF-11 | El sistema debe permitir enviar solicitud de amistad. | Alta | Desde el perfil de un usuario, se puede enviar solicitud creando un registro en friends con estado PENDING.
RF-12 | El sistema debe permitir aceptar o rechazar solicitudes de amistad. | Alta | El usuario receptor puede aceptar o rechazar, actualizando el estado a ACCEPTED o REJECTED.
RF-13 | El sistema debe mostrar la lista de amigos confirmados. | Alta | Existe una seccion con amigos listados con su display_name, avatar y acceso a su perfil.
RF-14 | El sistema debe impedir solicitudes o amistades duplicadas. | Media | Intentar reenviar solicitud a usuario con relacion PENDING o ACCEPTED devuelve error de conflicto 409.

2.4 Feed Social de Actividad Musical

RF-15 | El sistema debe mostrar un feed cronologico con reproducciones de amigos. | Alta | En la vista principal, el usuario ve eventos ordenados por played_at descendente de sus amigos y del propio usuario.
RF-16 | Cada item del feed debe mostrar: amigo, titulo, artista, caratula y timestamp. | Alta | Se muestran esos datos por evento; si no hay caratula se usa un placeholder.
RF-17 | El sistema debe actualizar el feed periodicamente sin recargar la pagina. | Media | El frontend revalida datos (React Query) cada cierto intervalo configurable o al recuperar el foco de la ventana.

RF-18 | El sistema debe permitir filtrar el feed por un amigo especifico. | Baja | Un filtro o dropdown permite seleccionar un amigo y ver solo su actividad aplicando filtro user_id.

2.5 Reacciones y Notas

RF-19 | El usuario debe poder reaccionar con emojis predefinidos a una reproduccion. | Alta | Al seleccionar un emoji, se guarda la reaccion para ese evento y usuario en la tabla reactions.
RF-20 | El sistema debe evitar reacciones duplicadas del mismo tipo por el mismo usuario y evento. | Media | Si el usuario ya reacciono con el mismo emoji al mismo playback_event, el sistema actualiza o ignora segun la politica acordada.
RF-21 | El usuario debe poder agregar una nota o comentario corto a una reproduccion. | Media | Al hacer clic en Agregar nota, puede escribir hasta 280 caracteres y la nota queda en la tabla notes asociada al evento.
RF-22 | El sistema debe mostrar el conteo de reacciones por tipo en cada item del feed. | Media | Se muestran conteos agrupados por emoji (ej. 3 veces corazon) y al pasar el cursor se pueden ver quienes reaccionaron.
RF-23 | El usuario debe poder ver notas asociadas a un evento. | Baja | Un boton Ver notas abre un panel con lista de notas (usuario, fecha, contenido) para ese playback_event.

2.6 Mensajeria 1 a 1 (General y Contextual)

RF-24 | El usuario debe poder iniciar una conversacion general 1 a 1 con un amigo. | Alta | Desde el perfil o lista de amigos, se puede iniciar conversacion creando un registro en conversations con origin_playback_event_id NULL.
RF-25 | El usuario debe poder iniciar conversacion desde el feed con Responder a cancion. | Alta | Al hacer clic, se abre chat con ese amigo y si no existia conversacion, se crea con origin_playback_event_id del evento del feed.
RF-26 | El sistema debe soportar envio de mensajes de texto en tiempo real via WebSockets. | Alta | Al enviar un mensaje, el receptor lo ve en su UI en menos de 3 segundos si esta online; el mensaje queda en la tabla messages.
RF-27 | El sistema debe almacenar historial de mensajes por conversacion. | Alta | Al abrir una conversacion, se listan mensajes previos ordenados por created_at ascendente.
RF-28 | Conversaciones con origen musical deben mostrar la cancion de origen. | Media | En el encabezado del chat se muestra la caratula y nombre de la cancion que inicio la conversacion.
RF-29 | El sistema debe permitir mensajes sin contexto de cancion. | Alta | El campo playback_event_id en messages es nullable y el usuario puede chatear libremente sin referenciar canciones.
RF-30 | El sistema debe indicar mensajes leidos y no leidos. | Media | La tabla messages tiene campos read y read_at; la lista de conversaciones muestra contador de no leidos por conversacion.

3. Requisitos No Funcionales (RNF)

RNF-01 | Rendimiento | P95 menor a 300 ms para APIs internas bajo carga normal (feed, amigos, mensajes).
RNF-02 | Rendimiento | Latencia WebSocket menor a 3 segundos entre envio y recepcion de mensajes bajo condiciones normales.
RNF-03 | Rendimiento | Nuevos eventos de amigos deben aparecer en el feed en menos de 60 segundos desde que Spotify los expone.
RNF-04 | Seguridad | access_token y refresh_token de Spotify no se exponen al frontend y se almacenan cifrados en PostgreSQL.
RNF-05 | Seguridad | Todas las rutas protegidas requieren JWT valido en header Authorization Bearer; expiracion configurable.
RNF-06 | Seguridad | Medidas contra OWASP Top 10: uso de ORM (Prisma contra SQLi), validacion de entradas, escaping en frontend, CSRF en flujos OAuth.
RNF-07 | Escalabilidad | Arquitectura stateless para API y WS, con posibilidad de escalar horizontalmente; DB con indices adecuados.
RNF-08 | Disponibilidad | Objetivo minimo 99.5% mensual excluyendo caidas de Spotify; degradacion controlada cuando Spotify este caido.
RNF-09 | Compatibilidad | Ultimas 2 versiones de Chrome, Edge, Firefox y Safari en desktop y mobile.
RNF-10 | Usabilidad | UI completamente funcional en pantallas desde 360px (movil) hasta 1920px (escritorio full HD).
RNF-11 | Observabilidad | Logs estructurados en JSON en el backend, metricas basicas de requests/errores y alertas ante errores 5xx.
RNF-12 | Mantenibilidad | ESLint + Prettier + TypeScript estricto; cobertura minima de tests unitarios del 70% sobre servicios criticos.

4. Arquitectura del Sistema

4.1 Diagrama General de Componentes

[DIAGRAMA 1 - AQUI VA EL DIAGRAMA DE ARQUITECTURA DEL SISTEMA (Mermaid flowchart LR mostrando: Cliente/Navegador con Next.js Frontend React, Servidor App con API REST Route Handlers / Servidor WebSocket / Modulo Auth Spotify / Servicios internos, Base de Datos PostgreSQL con todas las tablas, y API Externa Spotify)]

4.2 Estilo de Despliegue

Arquitectura monolitica logica: frontend y APIs REST corren en la misma app Next.js con Route Handlers. El servidor WebSocket corre junto al backend Node.js en el mismo proceso o como servicio separado segun el proveedor.

Despliegue recomendado con Docker:
- Contenedor app: Next.js + Node.js + WebSocket Server

- Contenedor db: PostgreSQL con volumen persistente
Alternativa en cloud: App en Vercel (Next.js) + PostgreSQL en Railway o Supabase + WebSocket en servidor dedicado Node (Railway o Render).

4.3 Flujo de Datos End-to-End

1. Usuario abre la app sin JWT valido: ve pantalla de login con boton Spotify.
2. Al pulsar el boton, el backend genera URL de autorizacion con PKCE y redirige a Spotify.
3. Spotify redirige a /api/auth/spotify/callback con code y state.
4. Backend valida state, intercambia code por tokens, guarda en streaming_accounts, crea/actualiza users, genera JWT y redirige al frontend.
5. Frontend va a /feed, llama a /api/feed con JWT. Backend obtiene amigos, consulta playback_events y opcionalmente refresca desde Spotify.
6. Usuario reacciona, agrega notas o inicia mensajes: llamadas REST al backend que persisten en PostgreSQL.
7. Para mensajes: cliente abre WebSocket, se autentica con JWT, envia y recibe mensajes en tiempo real; si receptor offline el mensaje queda guardado para carga posterior via REST.

5. Frontend

5.1 Stack Recomendado

- Framework: Next.js 15+ con App Router
- Lenguaje: TypeScript
- UI: React 18/19
- Estilos: Tailwind CSS + shadcn/ui para componentes base
- Estado servidor: TanStack Query (React Query) para fetch, cache y revalidacion
- Estado global ligero: Zustand o Context API
- WebSocket cliente: libreria ws o socket.io-client

5.2 Estructura de Carpetas

src/app/layout.tsx - Layout raiz (fuentes, providers, auth check)
src/app/page.tsx - Landing / pantalla de login
src/app/feed/page.tsx - Feed principal de actividad
src/app/profile/[id]/page.tsx - Perfil publico de usuario
src/app/messages/page.tsx - Lista de conversaciones
src/app/messages/[conversationId]/page.tsx - Vista de chat
src/app/settings/integrations/page.tsx - Integraciones Spotify
src/components/layout/ - AppShell.tsx, Sidebar.tsx, Topbar.tsx
src/components/auth/ - SpotifyLoginButton.tsx

src/components/feed/ - FeedList.tsx, FeedItem.tsx, ReactionBar.tsx, NotesPanel.tsx, NowPlayingBadge.tsx
src/components/friends/ - FriendSearch.tsx, FriendList.tsx, FriendRequestList.tsx
src/components/messages/ - ConversationList.tsx, ChatWindow.tsx, MessageBubble.tsx, MessageInput.tsx
src/components/common/ - Avatar.tsx, Button.tsx, TextInput.tsx, Badge.tsx, Modal.tsx, Spinner.tsx, EmptyState.tsx
src/hooks/ - useAuth.ts, useFeed.ts, useWebSocket.ts, useMessages.ts, useFriends.ts
src/lib/ - api.ts (cliente HTTP), ws.ts (cliente WebSocket), utils.ts

5.3 Rutas y Navegacion

/ - Si no autenticado: pantalla bienvenida con boton Spotify. Si autenticado: redirect a /feed
/feed - Vista principal: feed cronologico de actividad musical de amigos y del usuario
/profile/[id] - Perfil de usuario: avatar, nombre, estado de amistad, actividad reciente
/messages - Lista de conversaciones activas con ultimo mensaje y contador de no leidos
/messages/[conversationId] - Vista de chat: lista de mensajes, input y encabezado con cancion de origen si aplica
/settings/integrations - Estado de la vinculacion con Spotify, boton para reconectar si hay error de token

5.4 Layout Vista Feed (Escritorio 3 Columnas)

Header fijo horizontal: Logo MusicTwins (izquierda) | Buscador de usuarios (centro) | Avatar usuario con menu (derecha)

Columna izquierda (20%): Navegacion vertical (Feed, Mensajes, Settings) + Lista de amigos scrollable con avatars y nombres

Columna central (60%): Titulo Feed de amigos + FeedList (tarjetas una debajo de otra). Cada FeedItem contiene: avatar amigo, nombre, nombre cancion, artista, timestamp, caratula del album a la derecha, barra de reacciones con contadores, boton Agregar nota, boton Responder a cancion

Columna derecha (20%): Modulo Now Playing del usuario (si Spotify devuelve track actual) + Sugerencias de amigos (opcional)

5.5 Estados de Carga, Error y Vacio

Carga: Skeletons de FeedItem y ConversationList mientras se cargan los datos. Spinner en operaciones rapidas.
Error: Componente ErrorState en feed y mensajes con mensaje descriptivo y boton Reintentar que reactiva la query.

Vacio - Feed sin actividad: EmptyState con mensaje Aun no hay actividad, invita amigos a vincular su Spotify y CTA para buscar amigos.
Vacio - Sin conversaciones: EmptyState con mensaje Empieza un chat desde el feed al hacer clic en Responder a cancion.

6. Backend

6.1 Stack Recomendado

- Runtime: Node.js 20+
- Framework: Next.js Route Handlers (app/api/) para APIs REST
- Lenguaje: TypeScript
- ORM: Prisma 5+ con cliente para PostgreSQL
- WebSocket: libreria ws (ligera) o socket.io para el servidor en tiempo real
- Autenticacion: OAuth2 Spotify (PKCE) + JWT (jsonwebtoken)
- Validacion: Zod para validacion de inputs en todos los endpoints

6.2 Estructura del Proyecto (Backend)

src/app/api/auth/spotify/login/route.ts - GET: genera URL OAuth y redirige a Spotify
src/app/api/auth/spotify/callback/route.ts - GET: recibe code, intercambia tokens, crea usuario, genera JWT
src/app/api/auth/me/route.ts - GET: datos del usuario autenticado (requiere JWT)
src/app/api/friends/search/route.ts - GET: buscar usuarios por nombre
src/app/api/friends/requests/route.ts - GET/POST: listar y enviar solicitudes de amistad
src/app/api/friends/requests/[id]/route.ts - PATCH: aceptar o rechazar solicitud
src/app/api/friends/list/route.ts - GET: lista de amigos confirmados
src/app/api/feed/route.ts - GET: feed unificado y ordenado de actividad de amigos
src/app/api/reactions/route.ts - POST: crear reaccion a un playback_event
src/app/api/notes/route.ts - POST: crear nota a un playback_event
src/app/api/conversations/route.ts - GET/POST: listar y crear conversaciones
src/app/api/conversations/[id]/messages/route.ts - GET: historial de mensajes paginado
src/app/api/messages/read/route.ts - POST: marcar mensajes como leidos
src/app/api/spotify/now-playing/route.ts - GET: track actual del usuario en Spotify
src/app/api/spotify/recent/route.ts - GET: historial reciente de reproducciones
src/lib/auth/spotify.ts - helpers OAuth/PKCE (generacion URL, intercambio de tokens, refresh)
src/lib/auth/jwt.ts - helpers para firmar y verificar JWT
src/lib/db/prisma.ts - cliente Prisma singleton
src/lib/services/feedService.ts - logica de negocio del feed
src/lib/services/messageService.ts - logica de mensajes y conversaciones
src/lib/services/friendsService.ts - logica de amigos y solicitudes
src/lib/services/spotifyService.ts - llamadas a Spotify Web API con manejo de tokens

src/ws/server.ts - servidor WebSocket (handshake JWT, eventos MESSAGE_SEND/RECEIVED)

6.3 Especificacion de APIs REST

AUTH:
GET /api/auth/spotify/login | Sin auth | 302 redirect a URL de autorizacion Spotify | 400, 500
GET /api/auth/spotify/callback | Params: code, state | 302 redirect frontend con JWT en cookie | 400, 401, 500
GET /api/auth/me | JWT Bearer | 200: {id, displayName, avatarUrl, hasSpotifyLinked} | 401, 500

AMIGOS:
GET /api/friends/search?q= | JWT | 200: [{id, displayName, avatarUrl, isFriend, status}] | 400, 401, 500
GET /api/friends/requests | JWT | 200: [{id, fromUser, createdAt}] | 401, 500
POST /api/friends/requests | JWT, body: {targetUserId} | 201: {id, status: PENDING} | 400, 401, 404, 409, 500
PATCH /api/friends/requests/:id | JWT, body: {action: ACCEPT|REJECT} | 200: {id, status} | 400, 401, 404, 500
GET /api/friends/list | JWT | 200: [{id, displayName, avatarUrl}] | 401, 500

FEED:
GET /api/feed | JWT, query: friendId? limit? | 200: [{playbackEventId, user, track, artist, albumImage, playedAt, reactions}] | 401, 500

REACCIONES Y NOTAS:
POST /api/reactions | JWT, body: {playbackEventId, emoji} | 201: {id, playbackEventId, userId, emoji, createdAt} | 400, 401, 404, 500
POST /api/notes | JWT, body: {playbackEventId, content} | 201: {id, playbackEventId, userId, content, createdAt} | 400, 401, 404, 500

CONVERSACIONES Y MENSAJES:
GET /api/conversations | JWT | 200: [{id, user, lastMessage, unreadCount, originPlaybackEvent}] | 401, 500
POST /api/conversations | JWT, body: {targetUserId, originPlaybackEventId?} | 201: {id, user1Id, user2Id, originPlaybackEventId, createdAt} | 400, 401, 404, 500
GET /api/conversations/:id/messages | JWT, query: limit cursor | 200: [{id, senderId, content, playbackEventId, createdAt, read}] | 401, 404, 500
POST /api/messages/read | JWT, body: {conversationId} | 200: {updatedCount} | 401, 404, 500

SPOTIFY WRAPPER:
GET /api/spotify/now-playing | JWT | 200: {trackId, name, artist, album, imageUrl, isPlaying} | 401, 404, 500
GET /api/spotify/recent | JWT, query: limit? | 200: [{trackId, name, artist, album, imageUrl, playedAt}] | 401, 500

6.4 WebSocket: Especificacion

URL de conexion: wss://<host>/ws

Handshake (primer mensaje obligatorio del cliente):
  { type: AUTH, token: <jwt> }
Servidor responde: { type: AUTH_OK } o { type: AUTH_ERROR, message: string }

Enviar mensaje (cliente -> servidor):
  { type: MESSAGE_SEND, conversationId: string, toUserId: string, content: string, playbackEventId?: string }

Confirmacion al emisor:
  { type: MESSAGE_SENT, message: { id, conversationId, senderId, content, createdAt } }

Recepcion en tiempo real (servidor -> receptor si online):
  { type: MESSAGE_RECEIVED, message: { id, conversationId, senderId, content, createdAt } }

Notificacion de reaccion nueva (opcional):
  { type: REACTION_CREATED, data: { playbackEventId, emoji, fromUser } }

Si el receptor esta offline: el mensaje queda guardado en la tabla messages y se cargara por REST al abrir la conversacion.

6.5 Autenticacion y Autorizacion

OAuth 2.0 Spotify con PKCE: state aleatorio, code_verifier y code_challenge generados en el backend. Intercambio del code por tokens solo en el servidor.
JWT interno: contenido minimo { sub: userId, spotifyId, exp }. Firmado con JWT_SECRET del .env.
Reglas de autorizacion:
- Solo usuarios autenticados (JWT valido) pueden usar rutas protegidas.
- Para mensajes: se valida que el receptor sea amigo del emisor (estado ACCEPTED en tabla friends).
- Para reacciones y notas: solo amigos o el propio usuario pueden ver actividad en el feed.
- Tokens de Spotify: nunca salen del servidor, solo se usan internamente en spotifyService.ts.

7. Base de Datos

7.1 Stack Recomendado

Motor: PostgreSQL 14+ como base de datos relacional principal.
ORM: Prisma 5+ con migraciones controladas via prisma migrate.
Tipo de dato especial: JSONB para raw_metadata en playback_events (payload completo de Spotify).
UUIDs como claves primarias en todas las tablas para portabilidad y seguridad.

7.2 Diagrama ER (Modelo de Entidades)

[DIAGRAMA 2 - AQUI VA EL DIAGRAMA ER (Mermaid erDiagram con tablas: USERS, STREAMING_ACCOUNTS, FRIENDS, PLAYBACK_EVENTS, REACTIONS, NOTES, CONVERSATIONS, MESSAGES y todas sus relaciones)]

7.3 Descripcion de Tablas

USERS: id (uuid PK), spotify_id (text UNIQUE), display_name (text), email (text nullable), avatar_url (text nullable), created_at, updated_at

STREAMING_ACCOUNTS: id (uuid PK), user_id (uuid FK -> users), provider (text, ej spotify), provider_user_id (text), access_token_encrypted (text), refresh_token_encrypted (text), access_token_expires_at (timestamptz), created_at, updated_at

FRIENDS: id (uuid PK), user_id (uuid FK -> users), friend_user_id (uuid FK -> users), status (text: PENDING|ACCEPTED|REJECTED), created_at, updated_at. Constraint: (user_id, friend_user_id) UNIQUE para evitar duplicados.

PLAYBACK_EVENTS: id (uuid PK), user_id (uuid FK -> users), provider (text), track_id (text), track_name (text), artist_name (text), album_name (text), album_image_url (text nullable), raw_metadata (jsonb), played_at (timestamptz), created_at

REACTIONS: id (uuid PK), user_id (uuid FK -> users), playback_event_id (uuid FK -> playback_events), emoji (text), created_at. Constraint: (user_id, playback_event_id, emoji) UNIQUE.

NOTES: id (uuid PK), user_id (uuid FK -> users), playback_event_id (uuid FK -> playback_events), content (text, max 280 chars), created_at

CONVERSATIONS: id (uuid PK), user1_id (uuid FK -> users), user2_id (uuid FK -> users), origin_playback_event_id (uuid FK -> playback_events, nullable), created_at, updated_at. Constraint: (user1_id, user2_id) UNIQUE.

MESSAGES: id (uuid PK), conversation_id (uuid FK -> conversations), sender_id (uuid FK -> users), receiver_id (uuid FK -> users), content (text), playback_event_id (uuid FK -> playback_events, nullable), read (boolean DEFAULT false), read_at (timestamptz nullable), created_at

7.4 Indices Esenciales

playback_events(user_id, played_at DESC) - para feed de actividad por usuario ordenado por tiempo
friends(user_id, status) - para buscar amigos confirmados rapidamente
messages(conversation_id, created_at DESC) - para cargar historial de mensajes paginado
messages(receiver_id, read) WHERE read = false - para contar no leidos por usuario
users(spotify_id) UNIQUE - busqueda rapida por spotify_id en el callback OAuth

8. Integraciones y Servicios Externos

8.1 Spotify Web API (Fase 1 - Obligatorio)

Registro de app: en Spotify Developer Dashboard, crear app, obtener CLIENT_ID y CLIENT_SECRET, configurar redirect_uri apuntando a /api/auth/spotify/callback.

Scopes necesarios:
- user-read-currently-playing: para obtener el track en reproduccion
- user-read-recently-played: para el historial reciente
- user-read-email: para el email del usuario (si disponible)
- user-read-private: para display_name y avatar

Endpoints de Spotify usados:
- GET /me - datos del perfil del usuario
- GET /me/player/currently-playing - track actual
- GET /me/player/recently-played - historial reciente (limit parametro)
- POST /api/token - intercambio de code por tokens y refresh de access_token

Manejo de rate limits: implementar backoff exponencial simple, cachear resultados recientes (ej. currently-playing por 30s) para no superar limites de la API.

8.2 Apple Music (Fase 2 - Fuera del Alcance Inicial)

Se implementaria en una segunda fase usando MusicKit para la web y su proceso de autorizacion con tokens de desarrollador de Apple. No forma parte del alcance actual del documento.

9. Flujos Criticos (User Journeys)

9.1 Flujo: Login con Spotify

[DIAGRAMA 3 - AQUI VA EL DIAGRAMA DE FLUJO DE LOGIN CON SPOTIFY (flowchart TD: Usuario abre app -> tiene JWT valido? -> No: pantalla bienvenida -> clic Continuar con Spotify -> backend genera URL OAuth PKCE -> redirige a Spotify -> usuario autoriza? -> No: error y reintentar -> Si: callback con code y state -> backend valida e intercambia -> guarda tokens, crea usuario, genera JWT -> redirige a /feed)]

9.2 Flujo: Consultar Feed de Amigos

[DIAGRAMA 4 - AQUI VA EL DIAGRAMA DE FLUJO DEL FEED (flowchart TD: Usuario autenticado abre /feed -> GET /api/feed con JWT -> valida JWT -> obtiene amigos -> tiene amigos? -> No: devuelve lista vacia y muestra EmptyState -> Si: consulta playback_events en DB -> refresca algunos desde Spotify -> unifica y ordena por played_at -> devuelve lista -> UI renderiza tarjetas con canciones y reacciones)]

9.3 Flujo: Mensajeria en Tiempo Real (WebSocket)

[DIAGRAMA 5 - AQUI VA EL DIAGRAMA DE FLUJO DE MENSAJERIA WEBSOCKET (flowchart TD: Usuario abre /messages/conversationId -> cliente abre WebSocket wss://host/ws -> envia AUTH con JWT -> servidor valida JWT y asocia socket -> usuario escribe mensaje y presiona Enviar -> cliente envia MESSAGE_SEND -> servidor valida amistad y pertenencia a conversacion -> inserta en DB -> emite MESSAGE_SENT al emisor -> receptor online? -> Si: emite MESSAGE_RECEIVED al receptor -> UI receptor muestra mensaje nuevo -> No: mensaje queda como no leido para carga posterior via REST)]

9.4 Flujo: Vincular Cuenta de Streaming (UC1)

[DIAGRAMA 6 - AQUI VA EL DIAGRAMA DE FLUJO DE VINCULAR CUENTA (flowchart TD: Este flujo es parte del login inicial ya que la autenticacion es con Spotify directamente. Ver diagrama 3. Si el token caduca: usuario va a /settings/integrations -> clic Reconectar con Spotify -> mismo flujo OAuth desde el inicio -> tokens actualizados en streaming_accounts)]

9.5 Flujo: Reaccionar a Historial (UC3)

[DIAGRAMA 7 - AQUI VA EL DIAGRAMA DE FLUJO DE REACCION (flowchart TD: Usuario autenticado ve feed -> identifica cancion de un amigo -> clic en boton de reaccion -> se despliega panel de emojis -> usuario selecciona emoji -> POST /api/reactions con

playbackEventId y emoji -> backend valida JWT -> verifica que no sea reaccion duplicada -> inserta en reactions -> devuelve reaccion creada -> UI actualiza contadores del FeedItem)]

9.6 Flujo: Enviar Mensaje Contextual a Cancion (UC4)

[DIAGRAMA 8 - AQUI VA EL DIAGRAMA DE FLUJO DE MENSAJE CONTEXTUAL (flowchart TD: Usuario ve cancion en el feed de un amigo -> clic en Responder a cancion -> POST /api/conversations con targetUserId y originPlaybackEventId -> backend crea conversacion si no existe -> frontend redirige a /messages/conversationId -> encabezado muestra la cancion de origen -> usuario escribe mensaje -> MESSAGE_SEND via WebSocket -> flujo normal de mensajeria)]

10. Consideraciones de Implementacion

10.1 Variables de Entorno Requeridas

DATABASE_URL=postgresql://user:password@host:5432/musictwins
JWT_SECRET=clave-secreta-minimo-32-caracteres
SPOTIFY_CLIENT_ID=id-de-tu-app-en-spotify-dashboard
SPOTIFY_CLIENT_SECRET=secret-de-tu-app-en-spotify
SPOTIFY_REDIRECT_URI=https://tudominio.com/api/auth/spotify/callback
APP_BASE_URL=https://tudominio.com
NODE_ENV=production
LOG_LEVEL=info

10.2 Setup con Docker

docker-compose.yml con dos servicios:
- db: imagen postgres:14-alpine, variables POSTGRES_USER/DB/PASSWORD, volumen persistente musictwins_data, puerto 5432
- app: build desde Dockerfile en el root, dependencia de db, variables de entorno del .env, puertos 3000 (HTTP) y 3001 (WebSocket si separado)

Scripts utiles:
- npm run dev: desarrollo local con hot reload
- npm run prisma:migrate: aplicar migraciones en DB
- npm run prisma:studio: interfaz visual de la BD
- npm run build: build de produccion Next.js
- npm run test: ejecutar tests unitarios e integracion

10.3 Estrategia de Testing

Tests Unitarios (Jest + ts-jest):
- feedService: logica de construccion del feed con mocks de Prisma y Spotify
- messageService: insercion y recuperacion de mensajes
- friendsService: logica de solicitudes y estado de amistad
- spotifyService: helpers de refresh token y llamadas a la API con mocks de fetch
- jwt helpers: generacion y verificacion de tokens

Tests de Integracion (Jest + Supertest):
- /api/auth/spotify/callback: mockear llamadas a Spotify, verificar creacion de usuario y JWT
- /api/feed: verificar que solo retorna eventos de amigos del usuario
- /api/conversations y mensajes: flujo completo de creacion y lectura

Tests E2E (Playwright):
- Flujo completo: login -> ver feed -> reaccionar a cancion -> iniciar conversacion -> enviar mensaje
- Flujo de amigos: buscar usuario -> enviar solicitud -> aceptar -> verificar en lista

11. Riesgos y Supuestos

R-01 | Dependencia de disponibilidad de Spotify Web API | Alto | Media | Manejar errores y estados degradados con mensajes claros al usuario; cachear actividad reciente localmente.
R-02 | Cambios en politicas o limites de la API de Spotify | Alto | Baja | Revisar periodicamente la documentacion de Spotify; usar versiones estables de sus endpoints.
R-03 | Fuga de tokens de Spotify por mala gestion | Muy Alto | Baja | Almacenar cifrados, nunca exponer al frontend, auditoria de logs, rotacion periodica de claves.
R-04 | Escalabilidad de WebSockets al crecer la base de usuarios | Medio | Media | Disenar la interfaz WS para poder migrar a servicio gestionado (Pusher, Ably) sin cambiar el cliente.
R-05 | Complejidad de UX al combinar feed, reacciones, notas y mensajeria | Medio | Media | Diseno iterativo de UI; pruebas informales de usabilidad antes de finalizar componentes.
R-06 | Supuesto: todos los usuarios tienen cuenta de Spotify activa | Medio | Media | Si se detecta necesidad, planear soporte a otros proveedores en Fase 2.
R-07 | Supuesto: proveedor de hosting soporta Node + WebSockets persistentes | Alto | Baja | Verificar antes del despliegue; Vercel tiene restricciones para WS, considerar Railway o Render para el servidor WS.

12. Checklist de Implementacion

[ ] Crear repositorio y proyecto Next.js 15 con App Router, TypeScript y Tailwind CSS
[ ] Configurar Prisma con PostgreSQL y definir schema.prisma con todas las tablas del modelo
[ ] Ejecutar primera migracion y verificar esquema en DB
[ ] Registrar app en Spotify Developer Dashboard y obtener CLIENT_ID y CLIENT_SECRET

[ ] Implementar flujo OAuth Spotify: /api/auth/spotify/login y /api/auth/spotify/callback con PKCE
[ ] Implementar generacion y verificacion de JWT internos
[ ] Implementar APIs de amigos: search, requests (GET/POST), requests/:id (PATCH), list
[ ] Implementar spotifyService con refresh automatico de tokens
[ ] Implementar API de feed: GET /api/feed con logica de amigos y eventos
[ ] Implementar APIs de reacciones y notas
[ ] Implementar servidor WebSocket con handshake JWT y eventos de mensajeria
[ ] Implementar APIs de conversaciones y mensajes (REST para historial)
[ ] Construir UI escritorio: AppShell, Sidebar, Topbar, pagina de login
[ ] Construir componentes de feed: FeedList, FeedItem, ReactionBar, NotesPanel
[ ] Construir componentes de mensajeria: ConversationList, ChatWindow, MessageInput
[ ] Integrar TanStack Query para consumo de APIs con estados de carga y error
[ ] Integrar cliente WebSocket con hook useWebSocket
[ ] Ajustar estilos responsive para movil (min 360px)
[ ] Escribir tests unitarios para servicios criticos (cobertura minima 70%)
[ ] Escribir tests E2E para flujos principales con Playwright
[ ] Configurar Docker Compose y verificar despliegue local completo
[ ] Desplegar a entorno de staging y validar todos los flujos criticos
[ ] Revision de seguridad: OWASP checklist, cifrado de tokens, proteccion de rutas
[ ] Despliegue a produccion